



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**REDUCING ORGANIZATIONAL EXPOSURE TO
SOCIAL ENGINEERING ATTACKS USING AI-DRIVEN
RISK ASSESSMENT**

A CAPSTONE PROJECT REPORT

submitted in partial fulfilment of the requirements for the Course of

ITA1408 – Ethical Hacking

to the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE)**

Submitted by

S Deepadharshn (192472046)

Under the Supervision of

Dr S Parthiban

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Reducing Organizational Exposure to Social Engineering Attacks Using Ai-Driven Risk Assessment” has been carried out by S Deepadharshan (192472046) under the supervision of Dr S Parthiban and is submitted in partial fulfilment of the requirements for the current semester of the B.E Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

**Dr S Parthiban
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

COURSE FACULTY

**Dr S Parthiban
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.**

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

DECLARATION

I, S Deepadharshan (192472046) of the Computer Science and Engineering (Artificial Intelligence), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Reducing Organizational Exposure to Social Engineering Attacks Using Ai-Driven Risk Assessment” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:08/09/2026

**S Deepadharshan (192472046)
Signature of the Students with Names**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical
Sciences Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
S Deepadharshan (192472046)	Overall application development and integration; risk assessment, simulation, ML and dashboard implementation	Developed and integrated the Flask application, risk-assessment workflow, ML prediction, four simulation scenarios, database, recommendation engine and dashboard	Conducted functional testing, debugging, verification and interface refinement of the implemented modules	Contributed to preparation and organisation of the technical report, figures, tables and project documentation	70%
R Yadu Vamsi (192211703)	Supporting implementation, testing and documentation activities	Assisted with implementation and refinement of project components	Assisted in functional testing, verification and identification of implementation issues	Contributed to documentation and report preparation	30%
Total					100%

ABSTRACT

Social engineering represents a significant cybersecurity challenge because it exploits human decision-making rather than relying solely on technical vulnerabilities. Users may be exposed to deceptive emails, fraudulent technical-support requests, suspicious password-reset messages, and unsafe attachments that can lead to security incidents when handled incorrectly. Conventional security mechanisms primarily focus on detecting technical indicators of attacks, creating a need for complementary approaches that assess user behaviour and susceptibility.

This project presents Social Engineering Risk Assessment Using Machine Learning, implemented as a controlled web-based application named SecureRisk AI. The system provides an integrated platform for assessing user risk through structured assessment, machine-learning-assisted risk prediction, controlled social-engineering simulations, recommendations, and dashboard-based visualisation. Four simulation scenarios—Phishing Email, Fake IT Support, Password Reset, and Suspicious Attachment—are implemented to represent common social-engineering situations in a safe environment. User responses are processed by the application and incorporated into the risk-assessment workflow.

The system is developed using a modular Flask architecture with dedicated components for application routing, risk processing, simulation management, recommendation generation, machine-learning prediction, database operations, and user-interface presentation. A synthetic dataset is used to support the machine-learning workflow, while structured database storage enables assessment information to be processed and presented through the dashboard. Functional testing was performed to verify application execution, risk assessment, simulation workflows, result processing, recommendations, and dashboard functionality.

The developed system demonstrates the feasibility of combining behavioural simulation with machine-learning-assisted risk assessment in a controlled cybersecurity environment. The project provides a foundation for extending social-engineering assessment through additional scenarios, improved datasets, advanced analytics, and broader deployment capabilities.

Keywords: Social Engineering, Cybersecurity Risk Assessment, Machine Learning, Phishing, Security Awareness, Risk Prediction

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	10
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	16
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	20
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	27
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	37
	REFERENCES	41
	APPENDICES	42

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	SecureRisk AI Application Interface	11
Figure 4.1	SecureRisk AI Application Interface	27
Figure 4.2	SecureRisk AI System Architecture	28
Figure 4.3	Risk Assessment Interface	30
Figure 4.4	Risk Assessment Result	32
Figure 4.5	Social-Engineering Simulation Selection	34
Figure A.1	SocialEngineeringRiskAssessment Project Structure	42
Figure B.1	Flask Application Running in the Local Environment	43
Figure C.1	Available Social-Engineering Simulation Scenarios	43
Figure C.2	Phishing Email Simulation Interface	44
Figure C.3	Fake IT Support Simulation Interface	44
Figure C.4	Password Reset Simulation Interface	45
Figure C.5	Suspicious Attachment Simulation Interface	45
Figure D.1	Risk Assessment Testing Evidence	46
Figure D.2	Social-Engineering Simulation Testing Evidence	47
Figure E.1	SecureRisk AI Dashboard	48
Figure F.1	SocialEngineeringRiskAssessment GitHub Project Repository	48

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparative Analysis of Existing Solutions and Proposed System	18
Table 3.1	Risk Considerations and Mitigation Measures	25
Table 4.1	Major Software Components	28–29
Table 4.2	Functional Test Cases	30–31
Table 4.3	Summary of Implemented Results	32–33
Table 4.4	Objective Achievement Analysis	34
Table 4.5	Student Contribution	36
Table 4.6	Project Resource and Budget Summary	36
Table D.1	Testing Evidence Summary	45–46

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
ML	Machine Learning	—
URL	Uniform Resource Locator	—
HTTP	Hypertext Transfer Protocol	—
HTML	Hypertext Markup Language	—
CSS	Cascading Style Sheets	—
JS	JavaScript	—
SQL	Structured Query Language	—
CSV	Comma-Separated Values	—
DB	Database	—
F1	F1-Score	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Social engineering remains a significant cybersecurity challenge because it exploits human decision-making rather than relying exclusively on technical vulnerabilities. Attackers commonly use deceptive communication, urgency, impersonation, misleading requests, and malicious links or attachments to influence users into performing unsafe actions. Phishing emails, fraudulent password-reset requests, fake technical-support messages, and suspicious attachments are representative examples of such attacks.

Traditional cybersecurity mechanisms primarily focus on protecting networks, endpoints, applications, and data. However, these controls may not adequately address situations where a legitimate user is manipulated into performing an unsafe action. Consequently, assessing human susceptibility and providing appropriate awareness mechanisms are important components of organizational cybersecurity.

The Social Engineering Risk Assessment project, implemented as SecureRisk AI, addresses this problem through a web-based defensive assessment platform. The system combines user risk assessment, machine-learning-based risk prediction, simulated social-engineering scenarios, and recommendation mechanisms within a single application. The simulations are designed to operate locally and do not send actual emails or collect real credentials.

The application provides multiple controlled scenarios, including phishing email, fake IT support, password-reset, and suspicious-attachment simulations. Users can interact with these scenarios and perform actions such as reporting suspicious requests or analysing warning indicators. The resulting interaction data can be used to assess risk and support security-awareness decisions.

The system is implemented as a modular Flask application consisting of application routes, database components, machine-learning components, simulation services, recommendation services, templates, and frontend resources. This modular structure allows individual

security-assessment functions to be developed and integrated while maintaining a common application interface.

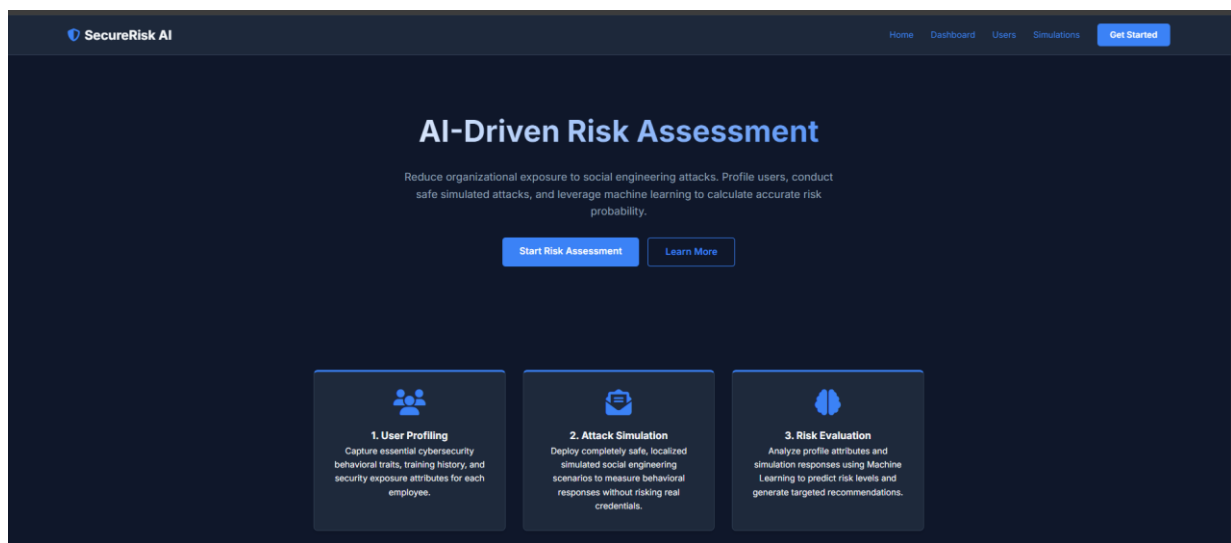


Figure 1.1: SecureRisk AI Application Interface

1.2 Need for the Project

The increasing use of digital communication has created more opportunities for attackers to exploit human behaviour through deceptive interactions. A security mechanism that only identifies technical vulnerabilities may therefore provide an incomplete representation of an organization's overall cybersecurity risk.

There is a need for a controlled assessment mechanism that can:

- identify user susceptibility to common social-engineering scenarios;
- provide realistic but safe security-awareness simulations;
- record user responses to simulated threats;
- estimate user-level cybersecurity risk;
- present risk information through an organizational dashboard; and
- provide recommendations based on identified risk conditions.

A further requirement is to conduct such assessments without exposing users to real malicious infrastructure or collecting genuine credentials. A locally executed simulation environment provides a safer approach for academic experimentation and controlled organizational assessment.

The proposed system addresses these requirements by integrating assessment, prediction, simulation, analysis, and reporting capabilities into one platform.

1.3 Problem Statement

Organizations commonly depend on employees as the final decision-making layer when handling emails, account requests, technical-support messages, links, and attachments. A user who cannot reliably distinguish legitimate communication from social-engineering attempts may unintentionally increase organizational cybersecurity risk.

Existing security mechanisms may detect malicious technical indicators, but they do not necessarily provide a unified mechanism for evaluating individual user behaviour across different social-engineering scenarios. In addition, conducting realistic awareness exercises using actual phishing infrastructure introduces unnecessary security, privacy, and operational risks.

Therefore, the engineering problem addressed by this project is:

To design and implement a controlled web-based cybersecurity risk-assessment system that evaluates user responses to simulated social-engineering attacks, applies machine-learning-based risk assessment, and presents the resulting risk information and recommendations through an integrated dashboard without using real credentials or external attack infrastructure.

1.4 Project Aim

The primary aim of the project is to develop a web-based Social Engineering Risk Assessment system that enables controlled simulation of common social-engineering scenarios and uses the resulting assessment information to support user-level and organizational cybersecurity risk evaluation.

The system is designed to combine behavioural assessment with machine-learning-based prediction and provide an accessible interface through which administrators can examine risk levels and simulation outcomes.

1.5 Project Objectives

The objectives of the project are:

1. To develop a web-based platform for assessing social-engineering-related cybersecurity risk.
2. To implement controlled simulations representing common social-engineering scenarios.
3. To provide users with opportunities to identify, analyse, or report suspicious requests.
4. To integrate a machine-learning component for risk prediction.
5. To store and process assessment information using a structured database.
6. To provide risk visualization through an organizational dashboard.
7. To generate recommendations based on identified cybersecurity risk conditions.
8. To maintain a safe simulation environment without transmitting real emails or collecting genuine credentials.
9. To test the integrated application and verify the functionality of its major components.

1.6 Scope of the Project

The scope of the project covers the development of a local web application for controlled social-engineering risk assessment. The major functional areas include user-profile management, risk assessment, machine-learning-based prediction, simulation execution, result processing, dashboard visualization, and recommendation generation.

The implemented simulation environment includes four scenarios:

S. No.	Simulation Scenario	Assessment Focus
1	Phishing Email	Recognition of deceptive email communication
2	Fake IT Support	Identification of fraudulent technical-support requests
3	Password Reset	Recognition of suspicious account-reset requests
4	Suspicious Attachment	Identification of potentially unsafe attachment-based requests

The project focuses on defensive cybersecurity assessment. It does not involve sending real phishing emails, obtaining actual passwords, deploying malware, compromising external systems, or conducting unauthorized penetration testing.

The application is intended for controlled laboratory or organizational awareness environments where simulated interactions can be used to understand potential human-related cybersecurity risk.

1.7 Expected Engineering Outcome

The expected outcome is a functional and integrated cybersecurity risk-assessment platform capable of connecting user assessment, simulated social-engineering interactions, machine-learning-based prediction, data storage, and risk visualization within a single application.

The developed system is expected to provide:

- a structured web interface for conducting assessments;
- controlled social-engineering simulations;
- user interaction and response handling;
- machine-learning-assisted risk prediction;
- database-backed assessment records;
- organizational risk visualization;
- risk-based recommendations; and
- a safe environment for cybersecurity awareness assessment.

The engineering value of the system lies in its integration of human-behaviour assessment and computational risk analysis rather than treating social engineering solely as a technical attack-detection problem. The modular design also provides a foundation for extending the platform with additional simulation scenarios, improved predictive models, and more comprehensive assessment metrics.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted to study recent developments in social-engineering and phishing attack detection, with particular attention to machine-learning-based cybersecurity solutions. Since the proposed project combines user-oriented risk assessment with machine-learning-assisted analysis, the review focused on research that demonstrates how computational techniques can be used to identify phishing characteristics and improve cybersecurity decision-making.

Research papers published from 2023 onwards were considered to maintain the relevance of the review to current cybersecurity developments. Five studies were selected based on their relevance to phishing detection, social-engineering threats, machine-learning techniques, feature-based classification, and practical security frameworks. The selected studies were analysed according to their research objective, methodology, findings, limitations, and relevance to the proposed Social Engineering Risk Assessment system.

2.2 Review of Existing Work

Social engineering remains a significant cybersecurity concern because attackers frequently exploit human trust and decision-making to achieve their objectives. Phishing is one of the most common forms of social engineering and can involve deceptive websites, URLs, domains, messages, or requests intended to influence users into taking unsafe actions. Recent research has therefore increasingly explored machine-learning techniques for automated phishing detection.

Alnemari and Alshammari (2023) investigated the detection of phishing domains using machine-learning techniques. Their study demonstrates the potential of machine-learning models to identify characteristics associated with malicious domains. The work provides evidence that automated classification can assist cybersecurity systems in distinguishing potentially harmful domains from legitimate ones.

Mosa et al. (2023) examined different machine-learning techniques for detecting phishing URL attacks. Their study used more than 11,000 websites and analysed 30 website-related features. Neural Networks, Naïve Bayes, and AdaBoost were evaluated, with reported accuracies of 90.23%, 92.97%, and 95.43%, respectively. The study demonstrates the

importance of selecting suitable features and classification techniques for phishing detection. Adu-Manu and Ahiabale (2023) proposed a multi-layered phishing detection framework that combined information from PhishTank, blacklist, blocklist, and whitelist sources. The framework was implemented through a web application and browser extension and was evaluated using 10,000 URLs. The reported system accuracy was 98.16%. The research demonstrates that combining multiple security indicators can strengthen phishing detection. Kapan and Sora Gunal (2023) conducted a comprehensive evaluation of different classifiers and feature groups for phishing attack detection. Their research examined URL, HTTP, and other feature groups across multiple datasets and classifiers. The study found that URL- and HTTP-based features provided strong performance, while a decision-tree classifier achieved an F1-score of 0.99 in their evaluation. The findings highlight the importance of feature selection and classifier selection in machine-learning-based security systems. Haq et al. (2024) investigated deep-learning-based phishing URL detection using a one-dimensional convolutional neural network. Their work demonstrates the continued development of automated phishing detection using deep-learning approaches and large collections of phishing and legitimate URLs. The study reinforces the role of machine learning in improving automated detection capabilities. Overall, these studies demonstrate that machine learning can effectively support automated phishing detection. However, most of the reviewed approaches primarily analyse technical properties such as URLs, domains, HTTP characteristics, or website features. This leaves an important area for further development: evaluating the human response to social-engineering situations within a controlled environment.

2.3 Comparative Analysis

The reviewed solutions mainly focus on determining whether a digital artefact, such as a URL, domain, or website, is legitimate or malicious. These approaches are valuable for automated threat detection, but they do not directly measure how users respond when confronted with deceptive requests.

The proposed SecureRisk AI system takes a different approach by incorporating user behaviour into the assessment process. Instead of analysing only external technical characteristics, the system provides controlled social-engineering scenarios and evaluates the user's response. This allows the system to connect behavioural assessment with machine-learning-based risk evaluation.

Table 2.1: Comparative Analysis of Existing Solutions and Proposed System

Parameter	Existing Solutions	Proposed SecureRisk AI System
Primary objective	Detect phishing URLs, domains, or websites	Assess user susceptibility to social engineering
Main data	URL, domain, HTTP and website features	User assessment information and simulation responses
Human behaviour	Limited consideration	Directly assessed
Social-engineering simulation	Generally not included	Included through controlled scenarios
Machine learning	Used mainly for phishing classification	Used for risk prediction
Risk assessment	Focused on threat classification	Focused on user risk level
Recommendations	Limited or detection-oriented	Provides risk-based recommendations
Dashboard	Not central to most solutions	Integrated dashboard
Simulation scenarios	Usually absent	Four controlled scenarios
Safety model	Detection of external threats	Local and controlled defensive assessment

The comparison indicates that the proposed system complements rather than replaces conventional phishing-detection approaches. Existing research provides strong foundations for automated identification of malicious technical indicators, while the proposed system extends the assessment process towards the human factor.

2.4 Research / Engineering Gap

The literature review identifies several engineering gaps in existing solutions. The first gap is the strong emphasis on technical phishing detection. Most reviewed systems classify URLs, domains, websites, or communication-related features. Although such detection is important, it does not determine whether a particular user is capable of recognising and responding correctly to a deceptive situation.

The second gap is the limited integration of controlled user simulations with machine-learning-based risk assessment. A user may interact with a phishing message, fraudulent support request, password-reset request, or suspicious attachment even when technical

security controls are available. Therefore, understanding the user's response provides an additional dimension of cybersecurity assessment.

The third gap is the lack of an integrated workflow connecting user assessment, simulation, risk prediction, database storage, visualisation, and recommendations. Many existing research works focus primarily on the performance of a particular detection model rather than developing a complete user-oriented assessment platform.

Based on these observations, there is a need for a defensive system that combines technical assessment capabilities with controlled behavioural evaluation. Such a system should allow users to experience representative social-engineering situations without exposing them to real attacks, while providing meaningful risk information for further security improvement.

2.5 Proposed Contribution

The proposed SecureRisk AI system addresses the identified gap through an integrated web-based Social Engineering Risk Assessment platform. The system combines user assessment, controlled social-engineering simulations, machine-learning-assisted risk prediction, database management, dashboard visualisation, and recommendations within a single application.

Four controlled scenarios are incorporated into the platform: Phishing Email, Fake IT Support, Password Reset, and Suspicious Attachment. These scenarios represent common forms of social-engineering situations and allow users to make decisions in a safe environment. The system records the relevant responses and uses the resulting information as part of the risk-assessment workflow.

The machine-learning component provides automated support for risk prediction, while the database layer stores structured application and assessment information. The dashboard presents the resulting risk information through summary indicators and visual representations, allowing the assessment results to be interpreted more easily. The recommendation component provides security-oriented guidance based on the assessed risk condition.

The main contribution of the project is therefore the integration of behavioural assessment and controlled social-engineering simulation with machine-learning-assisted risk evaluation. This provides a broader assessment perspective than systems that focus only on detecting malicious URLs or domains. The proposed platform is designed as a defensive and controlled environment and does not require real phishing campaigns, real credential collection, or interaction with external attack infrastructure.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

The proposed Social Engineering Risk Assessment system was designed as a web-based defensive cybersecurity application that integrates user assessment, controlled simulations, machine-learning-assisted risk prediction, data storage, and result visualisation. The requirements were derived from the identified engineering problem and the functional capabilities implemented in the application.

3.1.1 Functional Requirements

The system shall provide the following major functions:

1. **User Management:** The system shall allow user information to be maintained and accessed for assessment purposes.
2. **Risk Assessment:** The system shall collect relevant assessment information and determine an appropriate risk level for the user.
3. **Machine-Learning-Based Prediction:** The system shall use the trained machine-learning model to support automated risk prediction.
4. **Social-Engineering Simulation:** The system shall provide controlled scenarios representing common social-engineering situations.
5. **Simulation Response Processing:** The system shall process user actions within simulations and generate corresponding assessment results.
6. **Recommendation Generation:** The system shall provide security recommendations based on the assessed risk condition.
7. **Database Management:** The system shall store relevant application and assessment information in a structured database.
8. **Dashboard Visualisation:** The system shall present overall risk information through summary indicators and graphical representations.
9. **Safe Execution:** The system shall operate within a controlled environment without conducting real-world phishing campaigns or collecting actual user credentials.

3.1.2 Non-Functional Requirements

The major non-functional requirements include:

- **Usability:** The interface should be understandable to users with minimal navigation complexity.
- **Maintainability:** The application should be modular so that individual components can be modified without redesigning the entire system.
- **Security:** User and assessment information should be handled within the controlled application environment.
- **Reliability:** The application should consistently process assessments and simulation responses.
- **Scalability:** The architecture should allow additional simulation scenarios and assessment features to be incorporated in the future.
- **Performance:** User interactions and assessment operations should provide reasonable response times for a local web application.
- **Safety:** Simulations should remain controlled and should not produce real security incidents.

3.2 Constraints and Applicable Standards

The engineering design was developed under several practical constraints. The system is implemented as a local Flask web application, which limits the current deployment scope to a controlled environment. The machine-learning component depends on the available training data and the features used during model development. Consequently, the predicted risk should be interpreted as an assessment generated by the implemented model rather than as a definitive measurement of real-world user behaviour.

Another constraint is the use of controlled simulations. The project intentionally avoids sending real phishing emails, collecting real passwords, deploying malware, or interacting with external attack infrastructure. This constraint is necessary to maintain the defensive and educational purpose of the system.

The system design also considers general cybersecurity principles relating to confidentiality, integrity, availability, privacy, access control, secure handling of information, and responsible security testing. These principles guide the design without implying formal certification or compliance with a specific standard.

3.3 Design Specifications

The system follows a modular architecture in which different responsibilities are separated into application routes, service components, machine-learning components, database

components, and presentation templates.

The major design components are:

- **Presentation Layer:** HTML/Jinja templates, CSS, and JavaScript provide the user interface.
- **Application Layer:** Flask manages application execution and routing.
- **Risk Assessment Layer:** The risk engine processes assessment information and determines risk-related outcomes.
- **Simulation Layer:** The simulation engine manages controlled social-engineering scenarios and user responses.
- **Recommendation Layer:** The recommendation engine generates security-oriented guidance.
- **Machine Learning Layer:** The trained model and prediction components support automated risk prediction.
- **Database Layer:** The database components manage structured user and assessment information.
- **Data Layer:** The synthetic dataset provides data for the machine-learning workflow.

The modular arrangement allows individual components to be developed and tested separately while maintaining their integration through the Flask application.

3.4 Alternative Solutions and Evaluation

Several approaches could be considered for developing a social-engineering risk assessment platform.

Alternative 1: Rule-Based Risk Assessment

A rule-based system could assign risk levels using predefined conditions. For example, specific user responses could be mapped directly to low, medium, or high-risk categories.

Advantages:

- Simple to implement.
- Easy to understand and interpret.
- Requires limited computational resources.

Limitations:

- Less flexible when assessment factors change.
- Difficult to represent more complex relationships between multiple variables.
- Requires manual modification of rules.

Alternative 2: Machine-Learning-Based Risk Assessment

A machine-learning approach can learn relationships between input features and risk outcomes from training data.

Advantages:

- Supports automated prediction.
- Can consider multiple input characteristics.
- Provides a foundation for future model improvements.

Limitations:

- Performance depends on the quality and representativeness of the training data.
- Requires model training and validation.
- Predictions may not always be directly interpretable.

Alternative 3: Integrated Simulation and Risk Assessment

An integrated approach combines user assessment, controlled simulations, machine-learning-based prediction, database storage, and recommendations.

Advantages:

- Addresses both technical and behavioural aspects.
- Provides a complete assessment workflow.
- Allows simulation responses to contribute to risk evaluation.
- Supports dashboard-based analysis and recommendations.

Limitations:

- More complex to design and implement.
- Requires integration between multiple application components.
- Requires careful handling of simulation safety and user data.

Considering the objectives of the project, the integrated approach was selected because it provides a broader engineering solution than a standalone classification or rule-based system.

3.5 Selected Design and Detailed Design

The selected design is based on a layered and modular architecture. The user interacts with the web interface, while Flask routes direct requests to the appropriate application functions. The service layer then performs risk assessment, simulation processing, or recommendation generation as required.

The machine-learning component operates as a separate part of the application and provides risk prediction through the trained model. Assessment and simulation information is handled through the database layer, allowing the system to retain structured information for subsequent analysis.

The dashboard acts as the primary reporting interface. It combines stored assessment information and calculated risk outcomes into summary indicators and visual representations. This design allows individual user-level assessment information to be connected with a broader overview of risk conditions.

The simulation component contains four controlled scenarios:

1. **Phishing Email**
2. **Fake IT Support**
3. **Password Reset**
4. **Suspicious Attachment**

Each scenario is designed to represent a different social-engineering technique. The user is required to interpret the presented situation and select an appropriate response. The resulting interaction is processed by the application and contributes to the assessment workflow.

Figure Placement

[FIGURE 3: SecureRisk AI System Architecture]

The architecture diagram should illustrate the relationship between the presentation layer, Flask application/routes, service layer, machine-learning components, database, synthetic dataset, and application outputs.

3.6 Trade-Off Analysis

The system design required several engineering trade-offs. A major trade-off was between simplicity and analytical capability. A purely rule-based system would have been easier to implement and explain, but integrating a machine-learning component provides greater scope for automated risk prediction and future model development.

Another trade-off involved realism and safety. Highly realistic social-engineering simulations may provide stronger behavioural assessment, but real-world attack mechanisms could introduce ethical and security risks. The project therefore uses controlled scenarios that represent realistic situations without sending actual malicious messages or collecting real credentials.

A further trade-off concerns functionality and maintainability. Integrating multiple features into one application increases system functionality but also increases implementation complexity. The use of separate route, service, database, and machine-learning components helps manage this complexity.

The project therefore prioritises a balance between functionality, safety, maintainability, and extensibility rather than focusing on a single performance characteristic.

3.7 Sustainability, Ethics, Safety, Risk and Security

3.7.1 Sustainability

The application is designed as a lightweight local web platform and can operate using commonly available computing resources. The use of synthetic data for model development also reduces dependence on large collections of sensitive real-world user information. The modular architecture supports future extension without requiring complete redevelopment of the application.

3.7.2 Ethical Considerations

Social-engineering simulations must be designed responsibly because they imitate deceptive security situations. The proposed system is intended strictly for defensive assessment and education. The simulations are controlled and do not attempt to compromise real accounts or systems.

The project avoids collecting actual passwords or sensitive authentication credentials. Assessment results should also be interpreted responsibly, since a risk classification represents the behaviour observed within the assessment environment and should not be treated as a permanent judgement of an individual.

3.7.3 Safety Considerations

The simulations are implemented within the application rather than through real external communication infrastructure. This reduces the possibility of accidentally affecting external users, systems, or networks. The controlled approach also allows testing to be performed without introducing genuine phishing or malware-related threats.

3.7.4 Risk Considerations

Several risks were considered during development:

Risk	Potential Impact	Mitigation
Incorrect risk prediction	Misclassification of user risk	Use structured assessment data and model-based evaluation
Poor-quality training data	Reduced prediction reliability	Use structured synthetic data and controlled model training
Unsafe simulation design	Accidental exposure to real threats	Restrict scenarios to local controlled simulations
Database errors	Loss or inconsistency of assessment information	Use structured database models and controlled application operations
Interface errors	Incorrect user interaction	Test individual simulation and

		assessment workflows
Future feature expansion	Increased application complexity	Maintain modular routes and service components

3.7.5 Security Considerations

Security was considered throughout the application design. The system is intended to operate within a controlled environment and does not require real credentials for its simulation scenarios. Separation of application responsibilities also reduces unnecessary coupling between components.

The project follows a defensive security approach in which the system is used to identify potential user susceptibility and provide recommendations rather than to perform offensive activity. This distinction is essential for maintaining the ethical and safe use of the developed platform.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach and Resources

The Social Engineering Risk Assessment system was developed as a modular web application using Python and the Flask framework. The development approach focused on separating the user interface, application routing, business logic, machine-learning functionality, and data management into independent components. This structure improves maintainability and allows individual features to be tested without modifying the complete application.

The application is organised into separate modules for risk assessment, simulations, user management, database operations, machine-learning prediction, and recommendation generation. Flask provides the central application framework and manages the routing between the different functional components. Flask supports modular application organisation through blueprints, which is suitable for separating related application functionality.

The major development resources used in the project include Python, Flask, HTML, CSS, JavaScript, Jinja templates, SQLite-based database storage, and machine-learning libraries. A synthetic dataset was used as part of the machine-learning workflow, allowing model development without depending on sensitive real-world user information.

The project was developed and tested in a local environment. The application was executed using the Flask application module and accessed through a local browser interface. This approach provided a controlled environment for testing the social-engineering simulations and risk-assessment workflow.



Figure 4.1: SecureRisk AI Application Interface

4.2 Software Implementation and Modern Tools Used

The implementation follows a modular software structure. The main Flask application is located in `app/app.py`, while configuration-related functionality is maintained separately. Application routes are divided according to their responsibilities, including risk assessment, simulations, and user management.

The risk assessment module handles the user assessment workflow and communicates with the risk-processing components. The simulation module manages the four controlled scenarios implemented in the system: Phishing Email, Fake IT Support, Password Reset, and Suspicious Attachment.

Figure 4.2: SecureRisk AI System Architecture

The risk engine is responsible for processing risk-related information, while the recommendation engine provides security-oriented guidance based on the resulting assessment. The simulation engine controls the simulation workflow and response processing. The machine-learning functionality is implemented through dedicated training and prediction components. The trained model is stored as `risk_model.pkl`, allowing the application to load the previously trained model for prediction rather than requiring model training during every application execution. Persisting a trained machine-learning model is a standard deployment approach, although persisted model files should only be loaded from trusted sources because pickle-based model loading can execute arbitrary code.

The database layer consists of dedicated database and model modules, while the application interface is implemented using Jinja-based HTML templates, CSS, and JavaScript. The separation between these components reduces coupling and allows changes to the presentation layer without directly modifying the risk or simulation logic.

Table 4.1: Major Software Components

Component	Implementation	Purpose
Application	Flask / Python	Controls the web application
Routes	Flask route modules	Handles user requests and navigation
Risk Engine	Python service	Processes risk-related assessment information
Simulation Engine	Python service	Controls social-engineering simulations
Recommendation	Python service	Generates security recommendations

Engine		
ML Training	Python / ML libraries	Trains the risk prediction model
ML Prediction	Python	Loads the trained model and produces predictions
Database	SQLite / SQLAlchemy components	Stores structured application data
Frontend	HTML, CSS, JavaScript	Provides the user interface
Dataset	Synthetic CSV dataset	Supports the machine-learning workflow

4.3 Test Plan and Verification

Testing was performed to verify that the major functional components of the application operate correctly and that the complete workflow can be executed through the web interface. Testing focused primarily on functional verification of user navigation, risk assessment, simulation interaction, result generation, and dashboard presentation.

The application was first executed locally to verify that the Flask server started correctly. Individual pages were then accessed through the browser to confirm that the required templates and routes were functioning.

The four simulation scenarios were tested individually. Each scenario was opened through the simulation interface, user actions were performed, and the resulting response was checked. Particular attention was given to the interaction flow and the presentation of the analysis and result information.

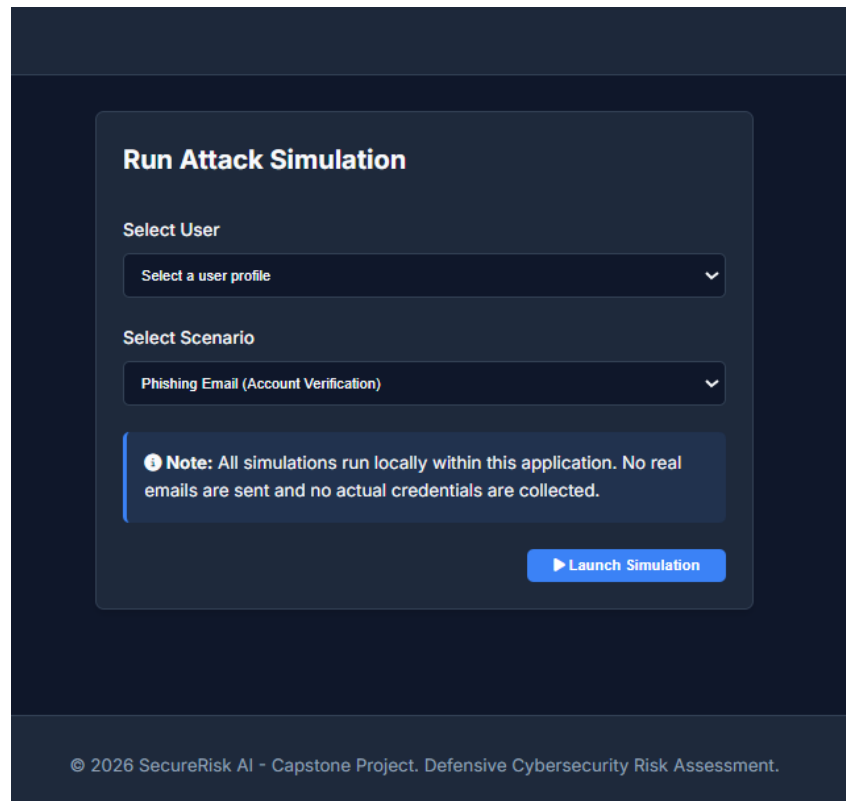


Figure 4.3: Risk Assessment Interface

The risk-assessment workflow was also tested by entering assessment information and verifying that the application generated a corresponding risk outcome. The dashboard was checked to ensure that stored assessment information was represented through the available summary indicators and visualisations.

Table 4.2: Functional Test Cases

Test ID	Test Function	Test Procedure	Expected Result	Status
T01	Application startup	Start the Flask application	Application starts successfully	Passed
T02	User interface	Open the main application page	Main interface loads correctly	Passed
T03	Risk assessment	Submit assessment information	Risk result is generated	Passed
T04	Phishing simulation	Complete phishing scenario	User response is processed	Passed
T05	Fake IT Support	Complete fake-support scenario	User response and analysis are displayed	Passed

T06	Password Reset	Complete password-reset scenario	Scenario result is generated correctly	Passed
T07	Suspicious Attachment	Complete attachment scenario	Scenario result is generated correctly	Passed
T08	Recommendations	View assessment recommendations	Relevant guidance is displayed	Passed
T09	Dashboard	Open dashboard	Risk information and visual summaries are displayed	Passed
T10	Database interaction	Perform assessment/simulation operations	Application data is processed through the database layer	Passed

The functional testing confirmed that the major implemented workflows could be executed successfully in the local application environment. The testing was focused on application functionality rather than claiming production-level security validation or large-scale performance validation.

4.4 Results

The implemented system successfully integrates the major components required for Social Engineering Risk Assessment. The application provides a unified workflow beginning with user interaction and assessment and extending through simulation, risk evaluation, recommendations, and dashboard presentation.

The four controlled social-engineering simulations were successfully implemented and tested. Each scenario provides a different type of deceptive situation, allowing the system to assess user responses across multiple social-engineering contexts.

The risk-assessment functionality provides a structured mechanism for evaluating users, while the machine-learning component supports automated risk prediction. The resulting information can then be presented through the dashboard to provide a broader overview of the assessment data.

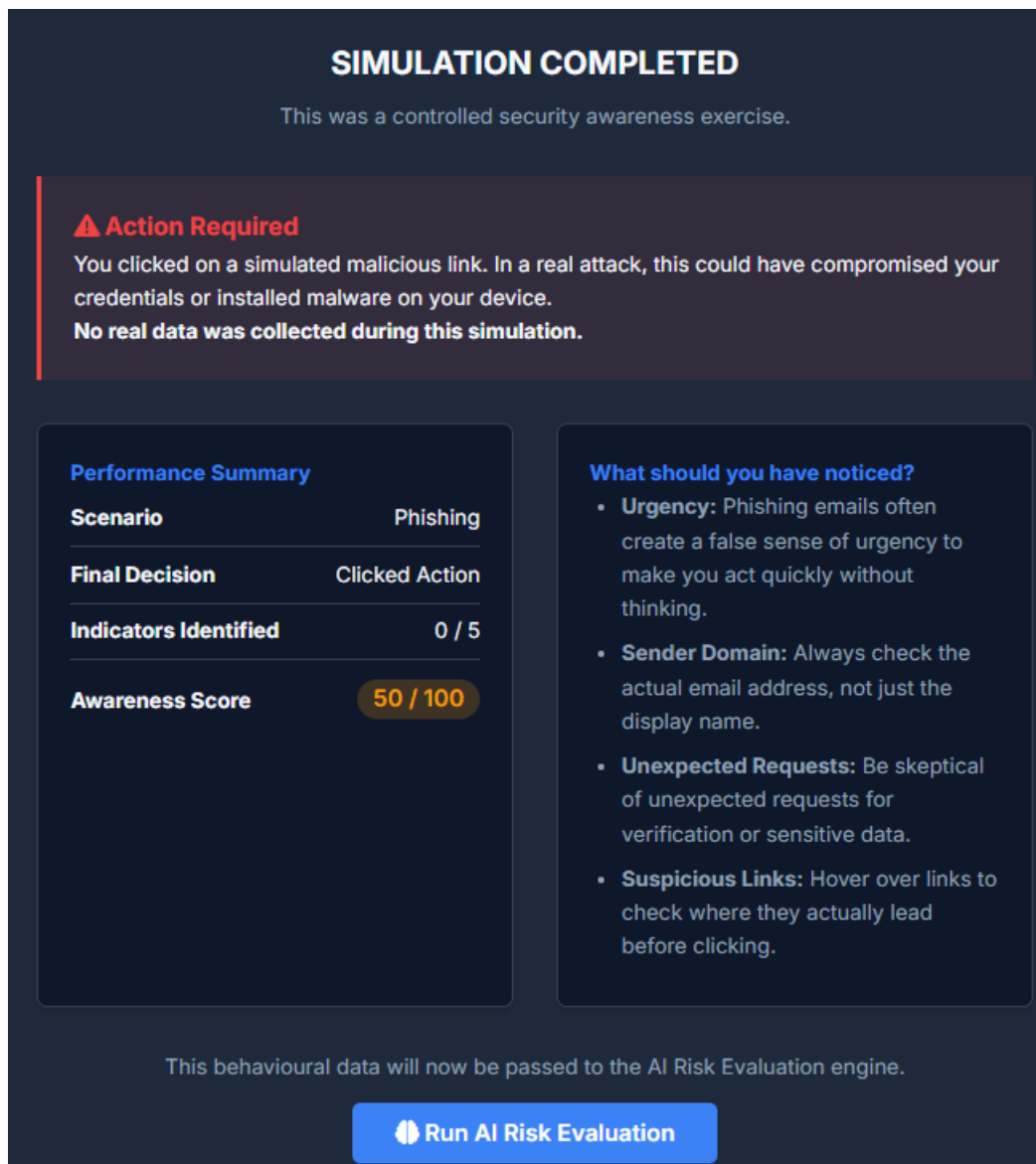


Figure 4.4: Risk Assessment Result

The application also successfully separates the different responsibilities into modular components. This makes the system easier to maintain and provides a foundation for adding additional simulation scenarios, assessment parameters, or machine-learning functionality in future development.

Table 4.3: Summary of Implemented Results

Feature	Implementation Result
Web application	Successfully implemented
User assessment	Successfully integrated
Machine-learning prediction	Integrated into risk-assessment workflow
Phishing Email simulation	Implemented and tested
Fake IT Support simulation	Implemented and tested

Password Reset simulation	Implemented and tested
Suspicious Attachment simulation	Implemented and tested
Recommendation mechanism	Implemented
Database integration	Implemented
Risk dashboard	Implemented
Local execution	Successfully verified

4.5 Data Analysis and Engineering Judgment

The developed system demonstrates that social-engineering assessment can be approached as a combination of behavioural evaluation and computational risk analysis. Rather than treating phishing as only a technical detection problem, the system considers how users respond when presented with representative social-engineering situations.

The four simulations provide different interaction contexts. This is useful because user susceptibility may not be identical across different attack types. A user who correctly identifies a phishing email may still respond incorrectly to a fraudulent technical-support request or suspicious attachment. Therefore, evaluating multiple scenarios provides a broader behavioural assessment.

The machine-learning component further supports the assessment workflow by providing automated prediction based on the available assessment data. However, the quality of any machine-learning prediction depends on the quality and representativeness of the training data. Since the current project uses a synthetic dataset, the resulting predictions should be interpreted as system-level assessment outputs rather than as validated predictions of real-world organisational behaviour.

From an engineering perspective, this limitation is important because it identifies a clear direction for future improvement: increasing the diversity and realism of training data while maintaining privacy and ethical safeguards.

Run Attack Simulation

Select User

Select a user profile

Select Scenario

Phishing Email (Account Verification)

Phishing Email (Account Verification)

Fake IT Support (Under Dev)

Password Reset Request (Under Dev)

Suspicious Attachment (Under Dev)

Figure 4.5: Social-Engineering Simulation Selection

4.6 Comparison with Existing Solutions and Achievement of Objectives

The developed system addresses the engineering gap identified in Chapter 2 by combining behavioural assessment with machine-learning-assisted risk evaluation. Existing phishing-detection approaches reviewed earlier primarily focus on technical indicators such as URLs, domains, and website characteristics. The proposed system adds a user-focused assessment layer through controlled simulations.

Table 4.4: Objective Achievement Analysis

Project Objective	Implementation	Achievement
Develop a web-based platform	Flask-based web application	Achieved
Implement social-engineering scenarios	Four controlled scenarios	Achieved
Evaluate user responses	Simulation response processing	Achieved
Integrate ML-based risk prediction	Trained model and prediction component	Achieved
Store assessment information	Database layer	Achieved
Provide risk visualisation	Dashboard with risk summaries and charts	Achieved
Generate recommendations	Recommendation engine	Achieved
Maintain a safe environment	Local controlled simulations	Achieved
Test the integrated system	Functional testing of major workflows	Achieved

The objectives were achieved at the implemented prototype level. The system provides the intended end-to-end workflow and demonstrates the feasibility of integrating simulation-based behavioural assessment with machine-learning-assisted risk evaluation.

4.7 Strengths and Limitations

4.7.1 Strengths

The primary strength of the system is its integrated architecture. Risk assessment, simulations, machine-learning prediction, recommendations, database management, and dashboard visualisation are provided within a single application.

Another important strength is the controlled nature of the simulations. The system can demonstrate common social-engineering situations without conducting real phishing campaigns or requiring genuine credentials. This makes the platform suitable for defensive assessment and educational use.

The modular implementation is also a significant engineering advantage. Separate route, service, database, and machine-learning components make the system easier to maintain and extend.

4.7.2 Limitations

The current system has several limitations. The machine-learning model is trained using synthetic data, which may not fully represent the complexity and diversity of real-world user behaviour. Consequently, the prediction output should not be considered equivalent to a validated real-world security assessment.

The current implementation also operates as a local web application rather than a large-scale deployed platform. Large-scale multi-user testing, cloud deployment, authentication hardening, and production-level monitoring were outside the current implementation scope.

In addition, the system currently provides four simulation scenarios. Although these cover several common social-engineering situations, a broader collection of scenarios would improve behavioural coverage.

4.8 Project Planning, Roles and Budget

The project was developed through a structured sequence of planning, implementation, debugging, testing, and refinement. Initial development focused on establishing the application structure and database components. This was followed by implementation of risk assessment, machine-learning prediction, simulations, recommendations, and dashboard functionality.

Testing and refinement were performed iteratively. Functional issues identified during execution were corrected and the affected components were retested. Interface-level improvements were also made to maintain consistency across the simulation windows and result displays.

The project involved two students, with work distributed according to the defined contribution levels.

Table 4.5: Student Contribution

Student	Contribution	Major Areas of Work
Student 1	70%	Application development, integration, risk-assessment workflow, simulation implementation, testing, debugging and documentation
Student 2	30%	Supporting implementation, testing, refinement and documentation activities

The project primarily used existing computing resources and open-source software technologies. Therefore, the implementation did not require significant additional hardware expenditure.

Table 4.6: Project Resource and Budget Summary

Resource	Requirement	Cost Category
Development computer	Required for development and testing	Existing resource
Python	Application development	Open source
Flask	Web framework	Open source
Machine-learning libraries	Model development and prediction	Open source
SQLite	Database management	Open source
Web browser	Application testing	Existing resource
Development environment	Coding and debugging	Existing resource

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Social Engineering Risk Assessment project successfully developed a controlled web-based platform for assessing user susceptibility to common social-engineering situations. The implemented system integrates user assessment, machine-learning-assisted risk prediction, controlled simulations, database management, recommendations, and dashboard-based visualisation within a single application.

The four implemented simulation scenarios—Phishing Email, Fake IT Support, Password Reset, and Suspicious Attachment—provide different situations through which user responses can be evaluated. This approach extends the assessment beyond conventional phishing detection by considering the behavioural aspect of cybersecurity.

The modular Flask architecture separates application routes, risk-processing services, simulation functionality, recommendation logic, machine-learning components, database operations, and presentation templates. Such separation supports maintainability and future extension. Flask officially supports modular application organisation through blueprints, making this type of separation suitable for larger applications.

Functional testing confirmed that the major application workflows could be executed successfully in the local environment. The system also maintains a controlled defensive scope by avoiding real phishing campaigns, genuine credential collection, malware deployment, and interaction with external attack infrastructure.

Although the current implementation is a prototype and uses synthetic data for the machine-learning workflow, it establishes a practical foundation for further development. The project demonstrates that combining behavioural simulation with automated risk assessment can provide a broader perspective on social-engineering exposure than relying only on technical phishing indicators.

5.2 Future Work

The developed system can be enhanced in several areas to improve its practical applicability and analytical capability.

5.2.1 Expansion of Simulation Scenarios

Additional scenarios can be introduced to cover a wider range of social-engineering techniques, including impersonation, QR-code phishing, social-media deception, fraudulent payment requests, and other context-specific attacks. Increasing scenario diversity would allow user behaviour to be evaluated across a broader range of situations.

5.2.2 Improved Training Data

The current machine-learning workflow uses synthetic data. Future development can incorporate larger and more representative datasets while maintaining appropriate privacy and ethical safeguards. Realistic anonymised assessment data could improve the ability of the model to represent different behavioural patterns.

5.2.3 Advanced Risk Analytics

Future versions could incorporate additional analytical techniques for identifying changes in user risk over time. Historical assessment results could be used to track improvement, identify recurring weaknesses, and provide more detailed risk trends.

5.2.4 Enhanced Dashboard

The dashboard can be extended with additional analytical features such as time-based risk trends, scenario-specific performance, department-level comparisons, and detailed assessment histories. These additions would improve the usefulness of the system for monitoring and decision-making.

5.2.5 Improved Authentication and Access Control

For deployment beyond a local prototype, stronger authentication, role-based access control, session security, audit logging, and secure configuration management should be incorporated.

These features would be particularly important if the application were used to manage assessment information for multiple organisations or user groups.

5.2.6 Deployment and Scalability

The current application is designed for local execution. Future work could deploy the system using a production-grade web architecture and database infrastructure. This would allow multiple authorised users to access the platform while providing appropriate monitoring, backup, and security controls.

5.2.7 Adaptive Recommendations

The recommendation engine could be extended to provide more personalised guidance based on repeated simulation performance. Instead of providing only risk-based recommendations, future versions could identify specific weaknesses and suggest targeted security-awareness activities.

5.3 Professional Learning and Individual Reflection

The project provided practical experience in developing a complete cybersecurity-oriented software system rather than implementing an isolated algorithm. The development process required integration of web application development, database management, machine-learning functionality, user-interface design, testing, debugging, and cybersecurity principles.

A significant learning outcome was understanding the importance of modular software design. Separating the application into routes, services, database components, machine-learning components, and presentation templates made it easier to identify and correct implementation issues. It also demonstrated how different engineering components must work together to produce a usable system.

The project also provided practical experience in handling cybersecurity simulations responsibly. Designing social-engineering scenarios required a balance between realism and safety. The final implementation therefore focuses on controlled simulations rather than real-world attack activity.

The development and testing process further strengthened problem-solving skills. Implementation issues encountered during development required examination of application behaviour, identification of the affected component, modification of the implementation, and subsequent verification. This iterative process provided practical exposure to software debugging and system-level testing.

From an engineering perspective, the project demonstrated that cybersecurity solutions should consider both technological and human factors. Technical detection mechanisms are important, but understanding how users respond to deceptive situations can provide additional information for security assessment. The developed platform therefore provides a foundation for continued work in human-centred cybersecurity and machine-learning-assisted risk assessment.

REFERENCES

1. Adu-Manu, K. S., & Ahiabile, R. K. (2023). Detecting phishing using a multi-layered social engineering framework. *Journal of Cyber Security*, 5(1), 13–32.
2. Alnemari, S., & Alshammari, M. (2023). Detecting phishing domains using machine learning. *Applied Sciences*, 13(8), 4649.
3. Haq, Q. E. u., Faheem, M. H., & Ahmad, I. (2024). Detecting phishing URLs based on a deep learning approach to prevent cyber-attacks. *Applied Sciences*, 14(22), 10086.
4. Kapan, S., & Sora Gunal, E. (2023). Improved phishing attack detection with machine learning: A comprehensive evaluation of classifiers and features. *Applied Sciences*, 13(24), 13269.
5. Mosa, D. T., Shams, M. Y., Abohany, A. A., El-kenawy, E. M., & Thabet, M. (2023). Machine learning techniques for detecting phishing URL attacks. *Computers, Materials & Continua*, 75(1), 1271–1290.

APPENDICES

APPENDIX A — PROJECT DIRECTORY STRUCTURE

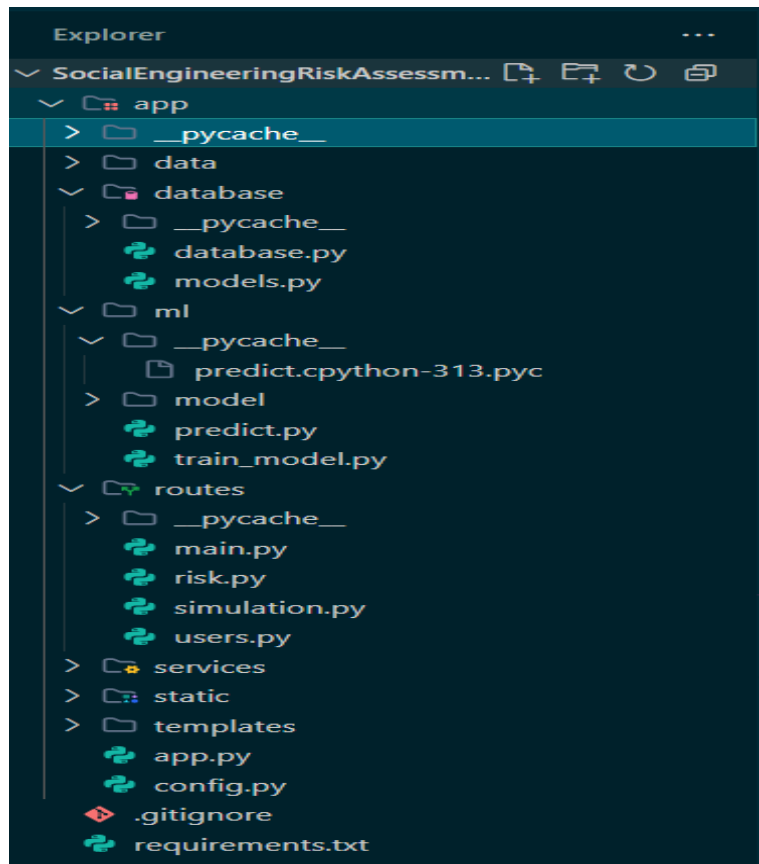


Figure A.1: SocialEngineeringRiskAssessment Project Structure

APPENDIX B — APPLICATION EXECUTION

This appendix documents the procedure used to execute the developed application.

The application can be executed from the project directory using the following command:

```
python -m app.app
```

After successful execution, the Flask development server provides the local application interface through:

```
http://127.0.0.1:5000
```

The application can then be accessed through a web browser for performing risk assessments and controlled social-engineering simulations.

```

PS D:\ITA1408\SocialEngineeringRiskAssessment> python -m app.app
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 623-555-500
127.0.0.1 - - [03/Sep/2026 09:50:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Sep/2026 09:50:22] "GET /static/css/style.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Sep/2026 09:50:22] "GET /favicon.ico HTTP/1.1" 404 -
* Detected change in 'C:\Users\Deepak\AppData\Local\Programs\Python\Python313\Lib\site-packages\flask\__init__.py', reloading
* Detected change in 'C:\Users\Deepak\AppData\Local\Programs\Python\Python313\Lib\site-packages\flask\app.py', reloading

```

Figure B.1: Flask Application Running in the Local Environment

APPENDIX C — SIMULATION SCENARIOS

The screenshot displays a web application titled "Run Attack Simulation". It features two main selection sections:

- Select User:** A dropdown menu with the placeholder text "Select a user profile".
- Select Scenario:** A dropdown menu currently showing "Phishing Email (Account Verification)". Below the dropdown, a list of available scenarios is visible:
 - Phishing Email (Account Verification)
 - Fake IT Support (Under Dev)
 - Password Reset Request (Under Dev)
 - Suspicious Attachment (Under Dev)

A blue progress bar is located at the bottom right of the interface.

Figure C.1: Available Social-Engineering Simulation Scenarios

CONTROLLED SIMULATION ENVIRONMENT

Inbox - Deepadharshan

Report Phishing

Analyze Message

Urgent Account Security Verification
From: IT Security <security-alert@company-training.test>
To: Deepadharshan <EMP001@company.com>
Date: Just now

Dear Deepadharshan,

Your account requires verification due to a recent security policy update. We have detected unusual login attempts from an unrecognized device.

Failure to verify your identity within 24 hours will result in temporary account restrictions to protect company data.

Verify Account Now

Thank you,
IT Security Department

Analyze Suspicious Indicators
Select the warning signs you noticed in this email:

☐ Urgent language ("Failure to verify...", "within 24 hours")

☐ Suspicious sender domain (@company-training.test instead of @company.com)

☐ Unexpected verification request (unusual login attempt warning)

☐ Suspicious action button/link

☐ Pressure to act immediately

Submit Analysis

Cancel

Figure C.2: Phishing Email Simulation Interface

CONTROLLED SIMULATION ENVIRONMENT

Inbox - Deepadharshan

Report Request

Analyze Message

Urgent IT Support Verification Required
From: IT Help Desk <it-support@company.test>
To: Deepadharshan <EMP001@company.com>
Date: Just now

Dear Deepadharshan,

Our monitoring system has detected unusual activity on your workstation. To ensure the security of your device, please verify your account information immediately.

Failure to comply may result in temporary suspension of your account.

Verify Account Now

IT Support Team

Analyze Suspicious Indicators
Select the warning signs you noticed in this message:

☐ Urgent language

☐ Suspicious sender domain

☐ Unexpected verification request

☐ Action button/link

Submit Analysis

Cancel

Figure C.3: Fake IT Support Simulation Interface

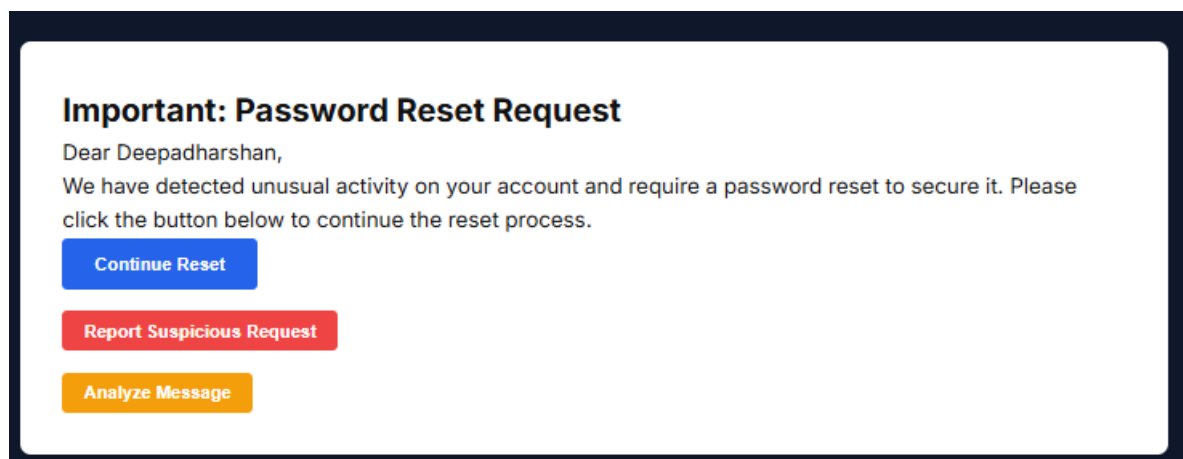


Figure C.4: Password Reset Simulation Interface

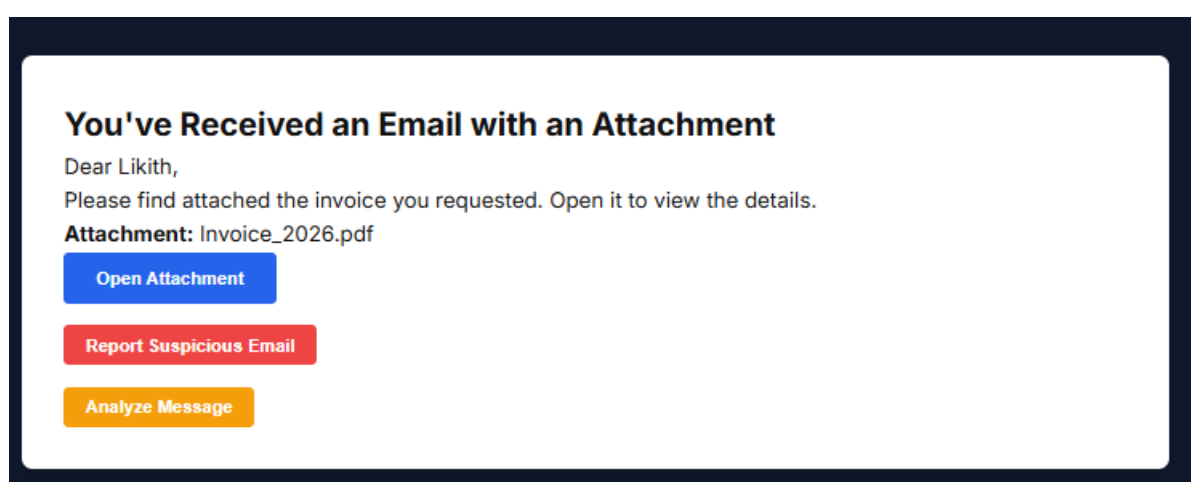


Figure C.5: Suspicious Attachment Simulation Interface

APPENDIX D — TESTING EVIDENCE

This appendix provides supporting evidence for the functional testing described in Chapter 4.

Table D.1: Testing Evidence Summary

Test Area	Verification
Application startup	Flask application successfully executed
Main interface	Interface successfully loaded
Risk assessment	Assessment workflow successfully executed
Phishing simulation	Scenario successfully tested
Fake IT Support	Scenario successfully tested
Password Reset	Scenario successfully tested
Suspicious Attachment	Scenario successfully tested
Result analysis	Simulation responses successfully processed

Recommendations	Recommendation output successfully displayed
Dashboard	Risk information successfully visualised

SIMULATION COMPLETED

This was a controlled security awareness exercise.

⚠ Action Required

You clicked on a simulated malicious link. In a real attack, this could have compromised your credentials or installed malware on your device.

No real data was collected during this simulation.

Performance Summary

Scenario	Suspicious Attachment
Final Decision	Clicked Action
Indicators Identified	0 / 5
Awareness Score	50 / 100

What should you have noticed?

- **Urgency:** Phishing emails often create a false sense of urgency to make you act quickly without thinking.
- **Sender Domain:** Always check the actual email address, not just the display name.
- **Unexpected Requests:** Be skeptical of unexpected requests for verification or sensitive data.
- **Suspicious Links:** Hover over links to check where they actually lead before clicking.

This behavioural data will now be passed to the AI Risk Evaluation engine.

 Run AI Risk Evaluation

Figure D.1: Risk Assessment Testing Evidence

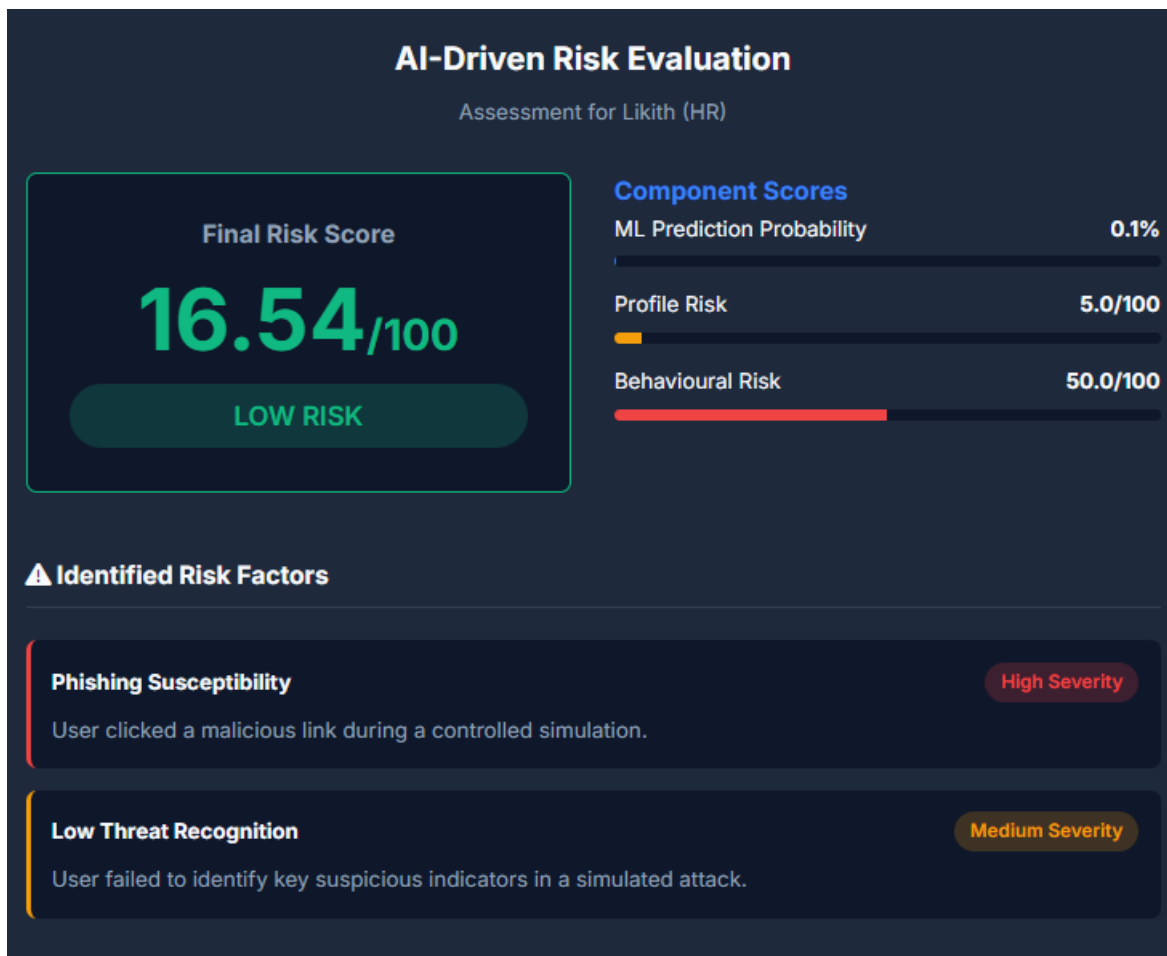


Figure D.2: Social-Engineering Simulation Testing Evidence

APPENDIX E — DASHBOARD AND RESULT VISUALISATION

The dashboard provides a consolidated representation of the assessment information generated by the system. It presents total users, risk-level summaries, average risk information, risk distribution, and department-wise risk information.



Figure E.1: SecureRisk AI Dashboard

APPENDIX F — GITHUB PROJECT REPOSITORY

The completed project was organised into the Mini-Projects GitHub repository as the SocialEngineeringRiskAssessment project. The repository contains the application source code, project configuration, requirements, machine-learning files, dataset, and supporting project structure.

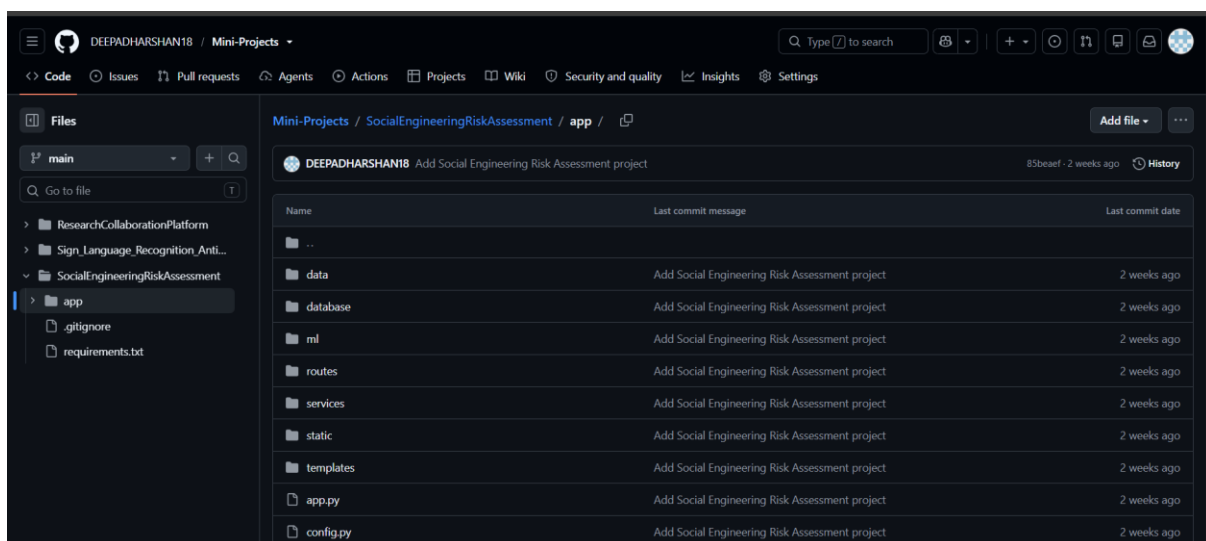


Figure F.1: SocialEngineeringRiskAssessment GitHub Repository